The Case for Coroutines

Document Number:	P4058R0
Date:	2026-03-15
Audience:	LEWG

Reply-to:	Vinnie Falco vinnie.falco@gmail.com

Table of Contents

Abstract

Revision History

- R0: March 2026 (post-Croydon mailing)

1. Disclosure

2. Two Models

3. Where `std::execution` Excels

4. The Price of Coroutines

5. The Stream Contract

- 5.1 Asio's `AsyncReadStream`
- 5.2 The C++20 Concept
- 5.3 What Changed

6. The Coroutine Dividend

- 6.1 Type Erasure Is Structural
- 6.2 The Operation State Is Not a Template
- 6.3 The Operation State Lives in the Socket
- 6.4 The Stream Can Be Type-Erased
- 6.5 The Stream Compiles Once
- 6.6 Synchronous and Asynchronous in One Abstraction
- 6.7 The Stream Is ABI-Stable
- 6.8 The Three Layers Emerge
- 6.9 The Frame Subsidizes Everything

7. What Trade-Offs?

8. Anticipated Objections

9. Conclusion

Acknowledgments

References

Abstract

C++20 coroutines provide five properties: type erasure through `coroutine_handle<>`, promise customization through `promise_type`, stackless frames that suspend and resume independently, symmetric transfer through `await_suspend` returning a handle, and compiler-managed state that survives across suspension. Each property was designed for generality - async patterns, lazy evaluation, generators. None was designed for I/O.

Combined, the five properties produce a substrate for serial byte-oriented I/O that resolves problems unique to C++: template explosion, compile-time cost, allocation control, and ABI instability. Type erasure plus compiler-managed state yields type-erased streams with zero per-operation allocation. Promise customization plus stackless frames yields frame allocator propagation before `operator new`. Symmetric transfer yields O(1) stack depth in deep coroutine chains. The synergy is not in any single property. It is in what the five produce together. This paper shows each property in working code, traces the causal chain from language mechanism to library design, and states the price.

This paper is one of a suite of six that examines the relationship between compound I/O results and the sender three-channel model. The companion papers are P4050R0[1], "On Task Type Diversity"; P4053R0[2], "Sender I/O: A Constructed Comparison"; P4054R0[3], "Two Error Models"; P4055R0[4], "Consuming Senders from Coroutine-Native Code"; and P4056R0[5], "Producing Senders from Coroutine-Native Code."

Revision History

R0: March 2026 (post-Croydon mailing)

- Initial version.

---

1. Disclosure

The author developed and maintains Corosio[6] and Capy[7] and believes coroutine-native I/O is the correct foundation for networking in C++. The author provides information, asks nothing, and serves at the pleasure of the chair.

The author regards `std::execution` as an important contribution to C++ and supports its standardization for the domains it serves well - GPU dispatch, heterogeneous execution, and compile-time work-graph composition among them. Nothing in this paper or its companions argues for removing, delaying, or diminishing `std::execution`. The author's position is narrower: that networking and stream I/O present a compound-result structure that the three-channel model was not designed to carry, and that this domain is better served by a coroutine-native facility that can coexist with senders and interoperate where the domains meet. Two models, each correct for its domain, is a stronger standard than one model asked to serve both.

---

2. Two Models

P3552R3[8] Section 9.4.1 [task.overview] defines the result:

> *"The `task` class template represents a sender that can be used as the return type of coroutines."*

One type. Two models. `std::execution::task` is a coroutine that is also a sender. Shipping it is an implicit endorsement of two async models in the C++ standard. The committee has already paid for two models.

On September 28, 2020, LEWG polled: "We must have a single async model for the C++ Standard Library." The result was no consensus (P2453R0[9], Poll 1 comment). On September 28, 2021, the Executors telecon polled again (P2453R0[9]):

> *"We believe we need one grand unified model for asynchronous execution in the C++ Standard Library, that covers structured concurrency, event based programming, active patterns, etc."*
>
> *SF:4 / WF:9 / N:5 / WA:5 / SA:1 - No consensus (leaning in favor).*

The "one model" premise was polled twice. It achieved consensus neither time. `task` ships two models.

This parallels the coroutine frame allocation: you pay once, you get everything that rides on it. Since the price is paid, the question is not whether to have two models but whether to get value from both.

---

3. Where `std::execution` Excels

The sender/receiver model is an achievement. The people who built it and deployed it have said so in their own words.

Eric Niebler described the philosophical foundation in 2020[10]:

> *"Structured concurrency brings the Modern C++ style to our async programs by making async lifetimes correspond to ordinary C++ lexical scopes, eliminating the need for reference counting to manage object lifetime."*

Child operations complete before their parents. Lexical scopes govern async lifetimes. The same discipline that makes synchronous C++ safe - RAII, deterministic destruction, nested scopes - extends to asynchronous code. This is the sender model's deepest conviction.

The practical motivation is equally clear. Niebler wrote in 2024[11]:

> *"Every library that exposes asynchrony uses a slightly different callback API. If you want to chain two async operations from two different libraries, you're going to need to write a bunch of glue code to map this async abstraction to that async abstraction. It's the Tower of Babel problem."*

One standard async abstraction solves the interoperability problem. Senders provide the common vocabulary.

Senders and coroutines are not either/or. Niebler framed this directly[11]:

> *"Returning a sender is a great choice: your users can await the sender in a coroutine if they like, or they can avoid the coroutine frame allocation and use the sender with a generic algorithm like `then()` or `when_all()`. The lack of allocations makes senders an especially good choice for embedded developers."*

The sender model provides zero-allocation pipelines through a specific design choice: `connect(sender, receiver)` produces an operation state that aggregates all data before `start()` is called. Niebler described the consequence[11]:

> *"We can launch lots of async work with complex dependencies with only a single dynamic allocation or, in some cases, no allocations at all."*

Separating construction from launch lets the pipeline aggregate all state before any work starts. The optimizer sees the full pipeline. This is possible because the operation state is parameterized on the receiver type - the compiler knows the complete type at every stage.

Senders also provide completion signatures as type-level contracts. The sender declares how it can complete. A type mismatch between pipeline stages is a compile error. The three-channel model - `set_value`, `set_error`, `set_stopped` - routes results by channel, and generic algorithms like `retry`, `when_all`, and `upon_error` dispatch on the channel without knowing the concrete sender type.

These are deployed at scale. P2470R0[12] documented the deployments: Facebook ("monthly users number in the billions"), NVIDIA ("fully invested in P2300... we plan to ship in production"), and Bloomberg (experimentation). GPU dispatch, infrastructure, HPC - the domains where compile-time work graphs, zero-allocation pipelines, and heterogeneous composition deliver their full value.

---

4. The Price of Coroutines

Coroutines are not free. Three costs are irreducible.

Frame allocation. When a function becomes a coroutine, the compiler moves everything that would normally live on the stack - every local variable, every function parameter, the suspension point that records where execution left off, and the awaitable machinery that manages resumption - into a heap-allocated block called the coroutine frame. Every coroutine that suspends must allocate this frame through `operator new`. The frame size is determined by the compiler. The caller cannot `sizeof` it, cannot stack-allocate it, cannot embed it in a struct. HALO (P0981R0[13]) can elide the allocation when the compiler proves the frame's lifetime is bounded by the caller's scope, but no compiler guarantees HALO. The recycling allocator (recycling_memory_resource[7]) amortizes the cost to a thread-local pool lookup - nanoseconds instead of microseconds - but the allocation still happens. Senders do not pay this cost. Sender operation states can be stack-allocated or embedded in the parent's operation state.

Opaque resume. The compiler cannot see through `std::coroutine_handle<>::resume()`. Every suspension point is an optimization barrier. The optimizer cannot inline across it. In tight inner loops this is measurable. This is the fundamental cost of type erasure through the handle. Senders do not pay this cost. Sender operation states are fully visible to the optimizer within a pipeline.

Reference lifetime hazard. Coroutine parameters are copied into the frame at the call site, but references are copied as references, not as values. A `const std::string&` parameter stores the reference in the frame. If the caller's string goes out of scope before the first suspension point, the reference dangles. Lambda captures by

reference have the same hazard. This is a correctness cost, not a performance cost. Google built `Co<T>` as immovable and prvalue-only specifically to prevent it (P3801R0[14]). Senders do not share this hazard in the same way - the operation state owns copies of everything passed through `connect`.

You pay the price once. A coroutine that does fifty reads pays one frame allocation and fifty zero-cost resumptions. The frame that you cannot avoid is the same frame that holds the operation state, the local variables, and the result. The type erasure that blocks inlining is the same type erasure that gives you `any_stream`, `task<T>` with one parameter, and non-template operation states. The price subsidizes everything in Section 6.

---

5. The Stream Contract

The committee has been trying to standardize networking since N1925[15] (2005). The contract that every attempt has been built on comes from Asio.

5.1 Asio's `AsyncReadStream`

The Boost.Asio documentation[16] defines `AsyncReadStream` as a named requirement with two operations:

operation	type	semantics
`a.get_executor()`	satisfying Executor requirements	Returns the associated I/O executor
`a.async_read_some(mb, t)`	determined by async operation requirements	completion signature `void(error_code ec, size_t n)`

Two operations. Twenty years. The contract has not changed. The Networking TS formalized it. The committee could not ship it. But the contract survived because it is correct.

5.2 The C++20 Concept

ReadStream[7] formalizes the same contract as a C++20 concept:

```
template<typename T>
concept ReadStream =
    requires(T& stream,
             mutable_buffer_archetype buffers)
    {
        { stream.read_some(buffers) }
            -> IoAwaitable;
        requires awaitable_decomposes_to<
            decltype(stream.read_some(buffers)),
            std::error_code, std::size_t>;
    };
```

`read_some` takes a buffer. The result satisfies `IoAwaitable`. The result decomposes to `(error_code, size_t)` via structured bindings. Nine lines.

The semantic requirements match Asio's: if `buffer_size(buffers) > 0` and `!ec`, then `n >= 1 && n <= buffer_size(buffers)` - at least one byte was read. If `ec`, then `n` is the number of bytes read before the I/O condition arose. I/O conditions are reported via the `error_code` component. Library failures (such as allocation failure) are reported via exceptions. The caller must ensure the buffer memory remains valid until the `co_await` expression returns.

5.3 What Changed

Two things vanished.

The completion token disappeared. In Asio, `async_read_some(mb, t)` takes a completion token `t` that determines the async model - callback, future, coroutine, or use_awaitable. In the coroutine model, the coroutine ***is*** the completion mechanism. There is no token. There is no choice. The coroutine suspends and resumes. The simplification is the trade-off.

`get_executor()` disappeared entirely. I/O objects do not carry executors. The caller provides the executor through `io_env` at `await_suspend` time. This resolves a long-standing Asio confusion where the I/O object has an executor and the completion token has a different executor, and the user must understand which one governs resumption. In the coroutine model there is one executor, it comes from the caller, and the I/O object never sees it.

What stayed: `read_some` takes a buffer, returns `(error_code, size_t)`. The named requirement became a concept. The language caught up to the contract.

---

6. The Coroutine Dividend

Asio made a different set of trade-offs. It supports callbacks, futures, coroutines, and completion tokens. That flexibility serves every async model a user might want. But the flexibility has a cost: the I/O operations must be templates, the operation states must be parameterized on the completion handler type, and the stream model cannot be type-erased without per-operation allocation. These are not defects. They are consequences of supporting every completion model simultaneously.

When you commit to coroutines as the only completion mechanism, you give up that flexibility. You cannot use callbacks. You cannot use futures. You cannot use completion tokens. That is a real cost. But the commitment unlocks optimizations and ergonomics that no other trade-off can deliver. The operation state becomes concrete. The stream becomes type-erasable. The library becomes separately compilable. The API becomes ABI-stable. None of these are possible when the completion mechanism is a template parameter.

The frame allocation is the cost every design avoids. This design accepts it. What follows is that the frame - the one cost that cannot be eliminated - subsidizes everything the ecosystem has been waiting for. Each consequence in the chain that follows is possible only because the previous one holds. Remove any link and the rest collapse. The chain is not a list of independent benefits. It is a single argument, nine steps long, and it begins with the frame.

6.1 Type Erasure Is Structural

Awaitable

```
struct read_awaitable
{
    bool await_ready();
    void await_suspend(
        std::coroutine_handle<> h);
        // caller erased
    io_result<size_t>
        await_resume();
};
```

Sender

```
template<class Receiver>
struct read_operation
{
    Receiver rcvr_;
        // caller stamped in
    void start() noexcept;
};
```

Same operation. Left: the caller is erased behind `coroutine_handle<>`. Right: the caller's type is stamped into the operation state as `Receiver`. The `<>` is the fork in the road. Everything in 6.2 through 6.9 follows from this difference.

`std::function` erases a callable. `coroutine_handle<>` erases a resumable. One is a library convention. The other is a language primitive. Both hide the concrete type behind a fixed interface. The difference is that `std::function` must allocate its own storage. `coroutine_handle<>` points to a frame the compiler already built.

When a coroutine `co_await`s an awaitable, the awaitable's `await_suspend` receives `std::coroutine_handle<>`. That handle is type-erased. The awaitable does not know what the caller's return type is, what the caller's promise type is, what local variables the caller holds, or where the caller will resume. The caller's entire identity - its variables, its suspension point, its future - is behind an opaque pointer. The awaitable sees one thing: a handle it can resume or destroy.

Any function that returns an awaitable gets structural type erasure of the caller for free. The I/O operation's state does not depend on who is waiting for it. A `read_op` struct is the same whether the caller is `task<int>`, `task<void>`, or any other coroutine type. The caller is erased. The operation state is concrete.

The cost is real. Because the handle is opaque, the compiler cannot optimize through `resume()`. It cannot inline the caller into the awaitable or the awaitable into the caller. The optimization barrier from Section 4 is the same mechanism that provides the type erasure. You do not get one without the other.

In the sender model, `connect(sender, receiver)` stamps the receiver's type into the operation state. The optimizer sees the full pipeline - it can inline across operation boundaries, eliminate dead code, and propagate constants through the entire chain. That is the strength for GPU pipelines (Section 3). The cost of that visibility is that the I/O operation's state depends on who is waiting for it. The same `async_read` connected to two different receivers produces two different operation state types.

Two models. One trades optimization for erasure. The other trades erasure for optimization. The sender model's causal chain diverges here. The receiver-parameterized operation state gives the optimizer full pipeline visibility - the strength documented in Section 3. But the chain that follows in 6.2 through 6.9 requires a concrete operation state. The sender model cannot enter it.

From here forward, the paper walks the coroutine chain alone.

6.2 The Operation State Is Not a Template

Windows (IOCP). overlapped_op[6] and read_op[6]:

```
struct overlapped_op : OVERLAPPED
{
    std::coroutine_handle<> h;
    capy::executor_ref      ex;
    std::error_code*        ec_out;
    std::size_t*            bytes_out;
    DWORD                   bytes_transferred;
};

struct read_op : overlapped_op
{
    WSABUF wsabufs[16];
    DWORD  wsabuf_count;
    DWORD  flags;
    win_socket_internal& internal;
};
```

Linux (epoll). epoll_op.hpp[6]:

```
struct epoll_read_op final
    : reactor_read_op<epoll_op> {};

struct epoll_write_op final
    : reactor_write_op<
        epoll_op, epoll_write_policy> {};
```

Not templates. Not parameterized on the caller. Known at library-build time. The same `read_op` serves every coroutine that reads from the socket, regardless of the caller's type.

6.3 The Operation State Lives in the Socket

win_socket_internal[6]:

```
class win_socket_internal
{
    connect_op conn_;
    read_op    rd_;
    write_op   wr_;
    SOCKET     socket_ = INVALID_SOCKET;
    int        family_  = AF_UNSPEC;
};
```

Three operation states. Members of the socket. Pre-allocated when the socket is created. Not allocated per-operation. 10,000 sockets means 10,000 `read_op` instances of one known type, allocated once, reused for every read.

6.4 The Stream Can Be Type-Erased

any_read_stream[7] type-erases any `ReadStream`. The erasure is on the awaitable, not the stream. The vtable[7] dispatches `await_ready`, `await_suspend`, `await_resume` through function pointers:

```
struct vtable
{
    void (*construct_awaitable)(
        void*, void*,
        std::span<mutable_buffer const>);
    bool (*await_ready)(void*);
    std::coroutine_handle<> (*await_suspend)(
        void*, std::coroutine_handle<>,
        io_env const*);
    io_result<std::size_t> (*await_resume)(void*);
    void (*destroy_awaitable)(void*) noexcept;
    std::size_t awaitable_size;
    std::size_t awaitable_align;
    void (*destroy)(void*) noexcept;
};
```

The awaitable storage is pre-allocated at construction time and reused for every `read_some` call. `read_some` constructs the real awaitable into cached storage, dispatches through the vtable, and destroys it on resume. One allocation at construction. Zero per-operation.

6.5 The Stream Compiles Once

Because `any_read_stream` has a fixed layout, a function accepting `any_read_stream&` goes in a `.cpp` file:

```
// dump.hpp
#include <boost/capy/task.hpp>
#include <boost/capy/io/any_read_stream.hpp>

capy::task<> dump(capy::any_read_stream& in);
```

```
// dump.cpp
#include "dump.hpp"
#include <boost/capy/buffers.hpp>
#include <iostream>

capy::task<> dump(capy::any_read_stream& in)
{
    char buf[1024];
    for(;;)
    {
        auto [ec, n] = co_await in.read_some(
            capy::mutable_buffer(buf, sizeof buf));
        if(ec)
            break;
        std::cout.write(buf, n);
    }
}
```

The header includes only Capy[7]. No platform headers. No Corosio[6]. No sockets. The `.cpp` compiles once. Consumers include the header and link. The stream behind `any_read_stream` could be a TCP socket, a TLS session, a file, or a test mock. Nothing recompiles.

This is also the Capy[7]/Corosio[6] split point. Capy delivers the abstract layer: `task<T>`, `any_read_stream`, `any_write_stream`, `any_stream`, buffer concepts, stream concepts, `when_all`, `when_any`, the frame allocator. Pure C++20. No platform dependency. Corosio delivers the platform layer: `tcp_socket`, `tls_stream`, timers, DNS, signals. Capy delivers value on its own - sans-I/O protocols, buffered streams, test mocks, all compile against Capy without Corosio. The architecture that made separate compilation possible is the same architecture that made the library split possible. One frame. Two libraries. Zero recompilation.

6.6 Synchronous and Asynchronous in One Abstraction

`co_await` checks `await_ready()` first. If the awaitable returns `true`, no suspension happens - the coroutine continues synchronously. A memory buffer, a test mock, a zlib decompressor, a base64 decoder - they all satisfy `ReadStream` by returning immediately-ready awaitables. The same `dump` function from Section 6.5 works whether the stream suspends for kernel I/O or returns instantly from a buffer. The algorithm does not know the difference.

A pipeline of `tcp_socket` -> `tls_stream` -> `decompression_stream` -> HTTP parser works regardless of which layers suspend. The synchronous layers pay zero suspension cost. No `is_async` flag. No separate sync API. One abstraction.

6.7 The Stream Is ABI-Stable

The vtable layout of `any_read_stream` does not change. Libraries compiled today work with new transports tomorrow. A `tls_stream` implementation compiled against OpenSSL 3.0 satisfies `ReadStream`. A future implementation compiled against a post-quantum TLS library will satisfy the same concept, plug into the same `any_read_stream`, and work with every library that was compiled against the old transport. No recompilation. No relinking. The forty-year-old contract - `read_some` takes a buffer, returns `(error_code, size_t)` - is the ABI.

6.8 The Three Layers Emerge

The user chooses the trade-off. io_stream[6], tcp_socket[6], native_tcp_socket[6]:

```
io_stream                       // abstract (Layer 3)
    |
tcp_socket                      // concrete (Layer 2)
    |
native_tcp_socket<Backend>      // native   (Layer 1)
```

Abstract (`io_stream`): virtual dispatch, ABI-stable, separately compiled. The compilation firewall. Business logic accepts `any_stream&` and never sees a platform header. Maximum compilation speed. Maximum insulation. The cost is virtual dispatch per I/O operation - nanoseconds against a microsecond syscall.

Concrete (`tcp_socket`): protocol-specific API - bind, connect, shutdown - still virtual dispatch, still separately compiled. Application code lives here. The full socket API without platform headers in the caller's translation unit.

Native (`native_tcp_socket<Backend>`): templated on the platform backend. Member function shadowing eliminates the vtable. Full inlining. Zero overhead. The cost is that the platform backend header is included and the code is in a header. Hot paths and benchmarks live here.

The user does not choose once for the whole application. Different layers coexist. A library accepts `any_stream&` (abstract). An application creates `tcp_socket` (concrete). A benchmark uses `native_tcp_socket<epoll>` (native). All three interoperate through the inheritance chain. The user gets the best of everything - compilation firewall where they want insulation, zero overhead where they want performance.

6.9 The Frame Subsidizes Everything

The coroutine frame paid for in Section 4 holds the local variables, the suspension point, and the result. `any_read_stream` works without per-operation allocation because the caller's frame already exists. The frame allocation you cannot avoid subsidizes the type erasure you want. This is the payoff for the price.

Additional properties that ride on the same frame:

- **Compile-time domain gate.** The two-argument `await_suspend(coroutine_handle<>, io_env const*)` is a deliberate trade-off. The alternative was a single-argument `await_suspend` that extracts the environment from the promise, costing one fewer parameter. The two-argument form was chosen because it buys compile-time enforcement: any awaitable that does not accept `io_env const*` is a type error inside an I/O task. Foreign awaitables that do not speak the I/O protocol are rejected by the compiler, not by a runtime mismatch. The pointer is the cost. The domain gate is the benefit. IoAwaitable[7].
- **Compound result preservation.** `auto [ec, n] = co_await sock.read_some(buf)` . Both values visible. No channel split. No data loss. The three-channel model routes results by channel. Compound results must choose a channel, losing data on the error path (P4053R0[2], P4054R0[3]).
- **Symmetric transfer.** `await_suspend` returns `coroutine_handle<>` . O(1) stack depth regardless of chain length.
- **One-parameter `task<T>`** . task.hpp[7]: `template<typename T = void> struct task` . One parameter. No Environment. The promise carries the environment.
- **Structured concurrency.** when_all[7] and when_any[7]. Both return `task<>` . Stop tokens propagate through `io_env` . Children complete before the parent resumes.

---

7. What Trade-Offs?

Six objections to the coroutine-native model are well-known. We concede every one.

1. Every coroutine that suspends pays a heap allocation. Senders do not.
2. `coroutine_handle<>::resume()` is an optimization barrier. Senders give the optimizer full pipeline visibility.
3. References captured in the coroutine frame can dangle. Sender operation states own copies.
4. Coroutines do not provide compile-time work graphs, static completion signature checking, or heterogeneous child composition.
5. Coroutines do not provide the composition algebra - `retry`, `when_all` sibling cancellation, `upon_error`.
6. Two async models in the standard library are harder to teach and maintain than one.

These are real costs. The question is what each model offers networking in return.

Coroutines	Senders
Type-erased streams, separate compilation, ABI stability	?

`connect(sender, receiver)` stamps the receiver type into the operation state. The same design decision that gives senders their optimizer visibility and their zero-allocation pipelines (Section 3) produces a template operation state that cannot live in the socket, cannot be type-erased without per-operation allocation, and cannot compile once.

The committee has been trying to ship networking since 2005. The template-heavy designs that reached the committee could not be separately compiled and were not ABI-stable. The committee did not ship networking.

---

8. Anticipated Objections

Q: Why not type-erase senders with `any_sender`?

A: `any_sender` type-erases the sender, not the receiver. `connect(any_sender, receiver)` still stamps the receiver type into the operation state. The operation state remains a template. The I/O object still cannot embed it, the stream still cannot compile once, and the ABI still depends on the receiver type. `any_sender` solves a different problem - it erases the sender's identity from the caller. The coroutine model erases the caller's identity from the I/O operation. Section 6.1 documents the distinction.

Q: Does `std::execution::task` not already bridge both models?

A: It does. Section 2 documents this. `task` is a coroutine that is also a sender. The question is not whether two models coexist - they already do. The question is whether the coroutine side of that coexistence carries I/O facilities that exploit the properties `coroutine_handle<>` provides: concrete operation states, type-erased streams, separate compilation, ABI stability. `task` bridges the models. It does not provide I/O.

Q: Two async models are harder to teach.

A: Section 7 concedes this. The cost is real. The paper does not argue the cost is zero. It documents what the coroutine model provides for networking (Sections 5 and 6) and what the sender model provides for its domains (Section 3), and observes that neither model can provide what the other provides.

---

9. Conclusion

The sender model makes design choices that give the optimizer full pipeline visibility, enable zero-allocation composition, and provide type-level completion contracts. These choices are correct for GPU dispatch, heterogeneous execution, and compile-time work graphs - the domains where `std::execution` is deployed at scale.

The coroutine model makes the opposite choice. `coroutine_handle<>` erases the caller. That erasure is the fork in the road. It forecloses pipeline optimization. It requires a frame allocation. But it produces a causal chain - concrete operation states, type-erased streams, separate compilation, ABI stability, a three-layer architecture - that resolves the properties the committee could not ship in twenty years of networking attempts.

Two models, each correct for its domain, is a stronger standard than one model asked to serve both.

Acknowledgments

The author thanks Chris Kohlhoff for Asio's stream model, buffer sequences, and executor architecture - twenty years of production deployment is the foundation this work builds on; Eric Niebler, Kirk Shoop, Lewis Baker, and their collaborators for `std::execution`; Gor Nishanov for the coroutine model's explicit support for task type diversity; Dietmar Kühl for `beman::execution` and P3552R3; Ian Petersen for identifying an asymmetry in an earlier draft and for confirming the equivalence between sender and coroutine dispatch; Ville Voutilainen for broadening the compound-result problem beyond I/O and for P2464R0; Jens Maurer for framing the design spectrum; Herb Sutter for identifying the need for tutorials and documentation; Jonathan Müller for confirming the symmetric transfer gap in P3801R0; Peter Dimov for the refined channel mapping; Klemens Morgenstern for Boost.Cobalt and the cross-library bridges; Steve Gerbino for co-developing the constructed comparison, bridge implementations, and Corosio; and Mungo Gill, Mohammad Nejati, and Michael Vandeberg for feedback.

References

1. P4050R0 - "On Task Type Diversity" (Vinnie Falco, 2026). https://wg21.link/p4050r0

2. P4053R0 - ''Sender I/O: A Constructed Comparison'' (Vinnie Falco, Steve Gerbino, 2026). https://wg21.link/p4053r0

3. P4054R0 - ''Two Error Models'' (Vinnie Falco, 2026). https://wg21.link/p4054r0

4. P4055R0 - ''Consuming Senders from Coroutine-Native Code'' (Vinnie Falco, Steve Gerbino, 2026). https://wg21.link/p4055r0

5. P4056R0 - ''Producing Senders from Coroutine-Native Code'' (Vinnie Falco, Steve Gerbino, 2026). https://wg21.link/p4056r0

6. cppalliance/corosio - Coroutine-native networking library. https://github.com/cppalliance/corosio

7. cppalliance/capy - Coroutine I/O primitives library. https://github.com/cppalliance/capy

8. P3552R3 - ''Add a Coroutine Task Type'' (Dietmar Kühl, Maikel Nadolski, 2025). https://wg21.link/p3552r3

9. P2453R0 - ''2021 October Library Evolution Poll Outcomes'' (Bryce Adelstein Lelbach, Fabio Fracassi, Ben Craig, 2022). https://wg21.link/p2453r0

10. Eric Niebler, ''Structured Concurrency,'' 2020. https://ericniebler.com/2020/11/08/structured-concurrency/

11. Eric Niebler, ''What are Senders Good For, Anyway?'' 2024. https://ericniebler.com/2024/02/04/what-are-senders-good-for-anyway/

12. P2470R0 - ''Slides for presentation of P2300R2'' (Eric Niebler, 2021). https://wg21.link/p2470r0

13. P0981R0 - ''Halo: coroutine Heap Allocation eLision Optimization'' (Gor Nishanov, 2018). https://wg21.link/p0981r0

14. P3801R0 - ''Concerns about the design of `std::execution::task`'' (Jonathan Müller, 2025). https://wg21.link/p3801r0

15. N1925 - ''A Proposal to Add Networking Utilities to the C++ Standard Library'' (Chris Kohlhoff, 2005). https://wg21.link/n1925

16. Boost.Asio AsyncReadStream requirements. https://www.boost.org/doc/libs/1_87_0/doc/html/boost_asio/reference/AsyncReadStream.html